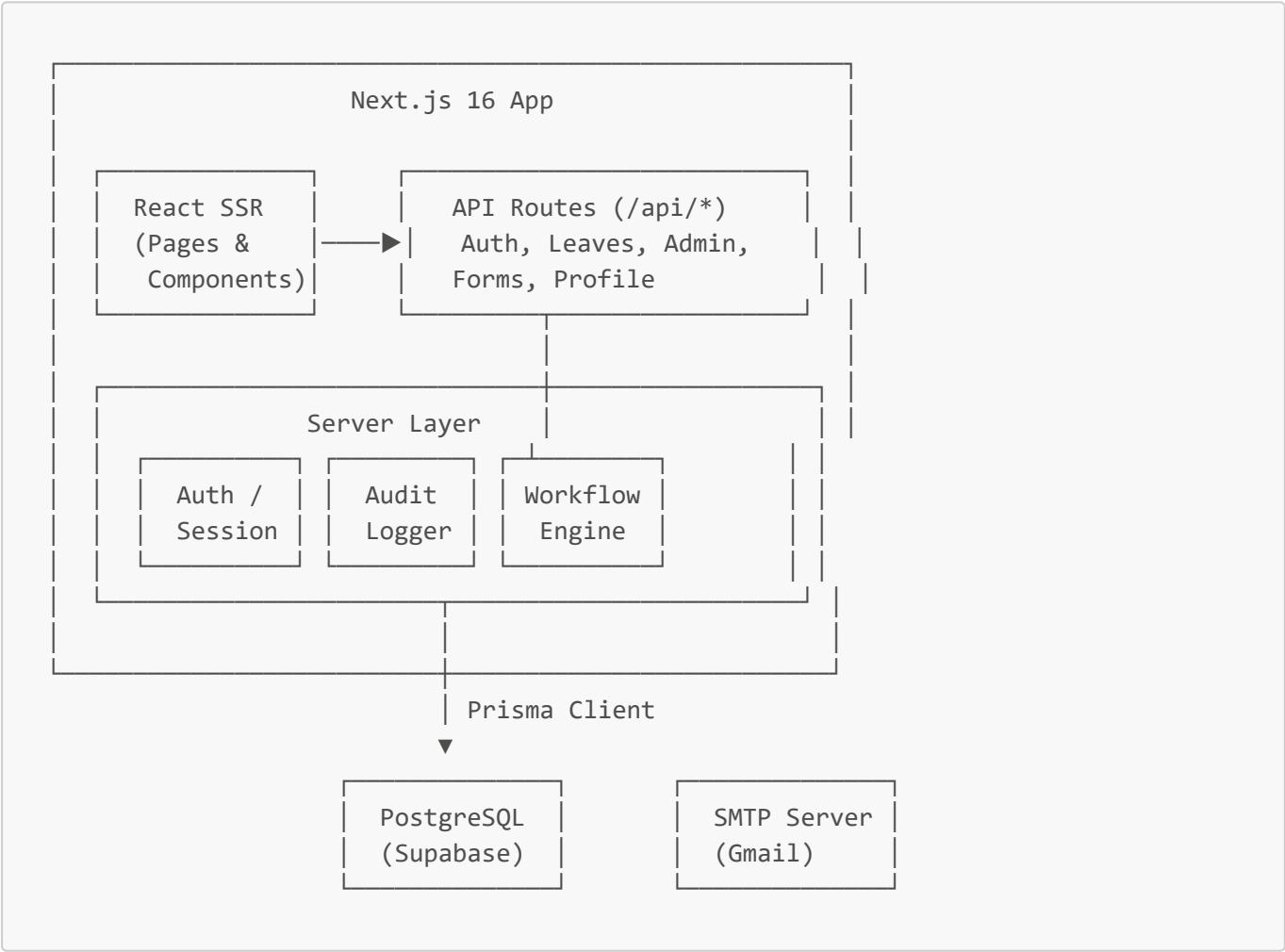


LeaveFlow (Leaves-IITRPR) — Complete Technical Architecture Overview

1. System Overview

Overall Architecture

- **Full-stack Next.js 16 monolith:** A single Next.js application that handles both server-rendered pages and API routes — no separate backend or microservice
- **Server-side rendering + API routes:** React Server Components render dashboard pages on the server; Next.js API routes (`/api/*`) handle all data mutations
- **PostgreSQL via Prisma ORM:** All persistent state lives in a Supabase-hosted PostgreSQL database accessed through Prisma Client
- **Email integration:** Nodemailer sends OTP codes and leave status notifications via SMTP (Gmail)



Components and Responsibilities

Component	Port	Responsibility
Next.js App	3000 (dev)	SSR pages, API routes, auth, leave workflows, admin panel, form builder

Component	Port	Responsibility
PostgreSQL	5432 (Supabase pooler: 6543)	Persistent data store (users, departments, roles, leave applications, approvals, audit logs, form templates)
SMTP (Gmail)	587	Sends OTP codes for authentication and leave status notification emails
Google OAuth	External	Optional Google sign-in integration

Complete Request Flow (Leave Submission)

1. User fills out an earned leave form in the browser (React client component with `react-hook-form` + Zod validation)
2. User enters OTP → digital signature is captured and verified server-side
3. Frontend sends `POST /api/earned-leave` with form payload
4. API route extracts session cookie (`lf_session`), calls `requireSessionActor()` to validate HMAC-signed session token
5. Server layer (`submitEarnedLeave`) validates payload, creates a `LeaveApplication` record with status `SUBMITTED`
6. Approval workflow engine generates sequential `ApprovalStep` records (HOD → Dean → Establishment, depending on leave type and role)
7. Audit logger records `SUBMIT_EARNED_LEAVE` event with IP address, user agent, and request metadata
8. Email mailer sends a styled HTML notification to the applicant confirming submission
9. API returns `{ ok: true, data: { id, referenceCode, status } }` with HTTP 201
10. Frontend navigates user to the "My Submissions" panel showing the new request

2. Tech Stack

Layer	Technology	Why Chosen
Framework	Next.js 16 (App Router)	SSR, API routes, file-based routing, React Server Components — single deployment unit
Language	TypeScript 5	Type safety across frontend and backend; Prisma types flow to components
UI	React 19 + Tailwind CSS 4	Component model with utility-first styling; Radix UI primitives for accessibility
Forms	react-hook-form 7 + Zod 4	Declarative validation with schema-first approach; <code>@hookform/resolvers</code> for Zod integration
Icons	Lucide React	Lightweight, tree-shakeable icon library
Typography	Space Grotesk (sans) + IBM Plex Mono (mono)	Google Fonts loaded via <code>next/font</code> for optimal loading

Layer	Technology	Why Chosen
Database	PostgreSQL (Supabase) + Prisma 6 ORM	Relational integrity for hierarchical approval chains; Prisma for type-safe queries and migrations
Auth	Custom HMAC session tokens + OTP (bcrypt)	Passwordless OTP-only auth via institute email; HMAC-SHA256 signed cookies
Email	Nodemailer	Sends OTP codes and styled leave status HTML emails via SMTP
PDF Export	html2canvas + jsPDF	Client-side PDF generation from rendered form HTML
Digital Signatures	signature_pad	Canvas-based signature capture with OTP verification
Client Encryption	Web Crypto API (AES-256-GCM)	Encrypts form auto-save data in localStorage with PBKDF2-derived keys
Linting	ESLint 9 + Prettier 3	Enforced via lint-staged + Husky pre-commit hooks
Deployment	Vercel / Supabase	Next.js on Vercel, PostgreSQL on Supabase (pooled via PgBouncer)

Key Alternatives Considered

Decision	Alternative	Why Current Choice Wins
PostgreSQL over MongoDB	MongoDB	Hierarchical approval chains need relational joins; Prisma migrations keep schema versioned
OTP-only over password auth	Email + password	Institute policy requires no passwords to manage; OTP via institute email is sufficient
Next.js over separate React + Express	React SPA + Express API	Single deployment, SSR for initial loads, shared TypeScript types, API routes colocated with pages
Prisma over raw SQL	Knex / TypeORM	Auto-generated TypeScript types, declarative schema, built-in migrations
Custom session tokens over NextAuth	NextAuth.js	Simpler OTP flow; no need for provider abstraction; full control over cookie/token lifecycle

3. Database Design

Database: PostgreSQL (Supabase, via Prisma ORM)

Tables

Department

Field	Type	Notes
id	String (CUID)	Primary key
name	String	e.g. "Computer Science & Engineering"
code	String	Unique index , e.g. "CSE", "ESTAB"
type	Enum: DepartmentType	ACADEMIC / ADMINISTRATIVE / ESTABLISHMENT / ACCOUNTS / DIRECTORATE
isActive	Boolean	Soft delete flag, default true

Role

Field	Type	Notes
id	String (CUID)	Primary key
name	String	Display name
key	Enum: RoleKey	Unique. FACULTY / STAFF / HOD / ASSOCIATE_HOD / DEAN / REGISTRAR / DIRECTOR / ACCOUNTS / ESTABLISHMENT / ADMIN
isApprover	Boolean	Whether this role participates in approval chains

User

Field	Type	Notes
id	String (CUID)	Primary key
email	String	Unique index , institute email
name	String	Full name
employeeCode	String	Unique , nullable (e.g. "IITRPR-F001")
designation	String	Job title
roleId	FK → Role	Role assignment
departmentId	FK → Department	Department membership
reportsToId	FK → User	Self-referential reporting chain (faculty → HoD → Dean → Director)
associateApproverId	FK → User	Delegated approver (HoD → Associate HoD)
isTeaching	Boolean	Teaching vs. non-teaching staff
isActive	Boolean	Account status

LeaveType

Field	Type	Notes
code	String	Unique. EL / SL / JR / EXIN / LTC / AIR
name	String	Unique. e.g. "Earned Leave"
category	String	Classification: Earned / Station / Post Leave / Foreign Travel / LTC / Travel Permission
maxDaysPerRequest	Int?	Per-request cap (60 for EL, 90 for EXIN)
requiresOfficeOrder	Boolean	Whether Establishment must issue an office order
requiresDirectorApproval	Boolean	Whether Director step is mandatory
requiresAccountsReview	Boolean	Whether Accounts must clear the request
requiresTravelPermission	Boolean	Whether travel permission is needed

LeaveApplication

Field	Type	Notes
id	String (CUID)	Primary key
referenceCode	String	Unique. Format: EL-2026-0001
applicantId	FK → User	Required
leaveTypeId	FK → LeaveType	Required
startDate / endDate	DateTime	Leave period
totalDays	Int	Computed duration
status	Enum: LeaveStatus	DRAFT / SUBMITTED / UNDER_REVIEW / APPROVED / REJECTED / RETURNED / CANCELLED
purpose	String	Reason for leave
destination	String?	Travel destination
exIndia / stationLeave / ltc	Boolean	Leave category flags
contactDuringLeave	String?	Emergency contact
accountsClearanceNeeded / directorApprovalNeeded	Boolean	Workflow routing flags
metadata	Json?	Flexible data (itinerary, logistics)
submittedAt / approvedAt / cancelledAt	DateTime?	Lifecycle timestamps

ApprovalStep

Field	Type	Notes
<code>leaveApplicationId</code>	FK → LeaveApplication	Required
<code>sequence</code>	Int	Step order (1, 2, 3...)
<code>actor</code>	Enum: WorkflowActor	APPLICANT / HOD / ASSOCIATE_HOD / DEAN / REGISTRAR / DIRECTOR / ACCOUNTS / ESTABLISHMENT
<code>status</code>	Enum: ApprovalStatus	PENDING / IN_REVIEW / APPROVED / REJECTED / ESCALATED / SKIPPED
<code>assignedToId</code>	FK → User	Specific user assigned to this step
<code>actedById</code>	FK → User	Who actually approved/rejected
<code>escalatedToId</code>	FK → User	Escalation target
<code>remarks</code>	String?	Approver comments

Constraint: `@@unique([leaveApplicationId, sequence])` — prevents duplicate steps

ActingHodAssignment

Field	Type	Notes
<code>hodId</code>	FK → User	The absent HoD
<code>actingHodId</code>	FK → User	The delegate
<code>assignedById</code>	FK → User	Who created the delegation
<code>startDate / endDate</code>	DateTime	Active period

Indexes: `[hodId, startDate, endDate]` and `[actingHodId, startDate, endDate]`

LeaveBalance

Field	Type	Notes
<code>userId</code>	FK → User	Employee
<code>leaveTypeId</code>	FK → LeaveType	Leave category
<code>totalAllocated / totalConsumed / totalEncashed</code>	Int	Balance tracking
<code>periodStart / periodEnd</code>	DateTime	Annual period

Constraint: `@@unique([userId, leaveTypeId, periodStart])`

AuditLog

Field	Type	Notes
<code>id</code>	String	UUID primary key (random, collision-safe)
<code>action</code>	String	e.g. SUBMIT_EARNED_LEAVE, APPROVE_LEAVE, LOGIN_OTP
<code>entityType</code> / <code>entityId</code>	String	What was acted upon
<code>referenceCode</code>	String?	Leave reference for cross-linking
<code>userId</code> / <code>userEmail</code> / <code>userName</code>	String?	Who performed the action
<code>ipAddress</code> / <code>userAgent</code>	String?	Request forensics
<code>details</code>	Json?	Structured metadata including <code>sourceIp</code> , <code>destinationHost</code> , <code>destinationIp</code>

Indexes: [`createdAt`], [`userId`, `createdAt`], [`ipAddress`, `createdAt`], [`referenceCode`], [`entityType`, `entityId`]

Immutability: PostgreSQL triggers (`auditlog_guard_insert` and `auditlog_guard_immutable`) enforce that only the system audit logger can insert rows, and no updates or deletes are allowed.

Notification

Field	Type	Notes
<code>userId</code>	FK → User	Recipient
<code>title</code> / <code>body</code>	String	Notification content
<code>type</code>	String	LEAVE / APPROVAL
<code>isRead</code>	Boolean	Read status

Index: [`userId`, `isRead`]

OtpToken

Field	Type	Notes
<code>email</code>	String	Target email
<code>tokenHash</code>	String	bcrypt hash of the 6-digit OTP
<code>expiresAt</code>	DateTime	Expiry (configurable, default 10 minutes)
<code>attempts</code>	Int	Rate limiting counter

OfficeOrder

Field	Type	Notes
<code>leaveApplicationId</code>	FK → LeaveApplication	Unique one-to-one link
<code>issuedById</code>	FK → User	Establishment officer
<code>orderNumber</code>	String	Unique order identifier
<code>documentUrl</code>	String?	Uploaded document

FormTemplate / FormSubmission / FormTaskInstance / FormSubmissionStepAction

Table	Purpose
<code>FormTemplate</code>	JSON-schema-driven configurable form definitions (admin-created, published)
<code>FormSubmission</code>	Submitted form data with workflow step tracking
<code>FormTaskInstance</code>	Task assignments for workflow steps (assigned to specific users)
<code>FormSubmissionStepAction</code>	Audit trail for each step action (ASSIGNED, COMPLETED, REJECTED)

Schema Design Decisions

- **Relational approval chain:** Approval steps are separate records linked to leave applications via `leaveApplicationId` + `sequence` — enables per-step tracking, escalation, and parallel approvals
- **Self-referential reporting chain:** `User.reportsToId` → `User` creates the organisational hierarchy (Faculty → HoD → Dean → Director) without a separate hierarchy table
- **Acting HoD delegation:** Time-bounded delegation table allows temporary authority transfer with full audit trail
- **Immutable audit log:** PostgreSQL triggers prevent any modification after insertion — critical for government/institutional compliance
- **Generic form system:** `FormTemplate` stores JSON schemas, allowing admins to create new form types without code changes
- **Leave balance per period:** Unique constraint on `[userId, leaveTypeId, periodStart]` prevents duplicate allocations and enables year-over-year tracking

Likely Expensive Queries

Query	Why Expensive	Location
Derived audit log aggregation	Merges <code>LeaveApplication</code> + <code>ApprovalStep</code> queries with text search, sorts, and pagination	<code>derived-audit.ts</code>
Approval step lookup with joins	Loads approval steps with nested <code>leaveApplication</code> → <code>applicant</code> → <code>leaveType</code> includes	<code>approvals/route.ts</code>
Admin statistics panel	Aggregates across all users, applications, and leave types	<code>admin-statistics-panel.tsx</code>

Query	Why Expensive	Location
User search by name/email	<code>contains</code> filter with <code>mode: insensitive</code> can't use standard B-tree indexes	<code>admin/users</code> API

4. APIs

Auth (`/api/auth`)

Method	Endpoint	Purpose	Auth
POST	<code>/request-otp</code>	Send 6-digit OTP to institute email	None
POST	<code>/verify-otp</code>	Verify OTP → create session token → set <code>lf_session</code> cookie	None
POST	<code>/logout</code>	Clear session cookie	Cookie
POST	<code>/remember-dashboard</code>	Save last visited dashboard path in cookie	Cookie
GET	<code>/google</code>	Google OAuth sign-in flow	None

Leave Applications

Method	Endpoint	Purpose	Auth
POST	<code>/api/earned-leave</code>	Submit an earned leave application	Session
POST	<code>/api/station-leave</code>	Submit a station leave request	Session
POST	<code>/api/joining-report</code>	Submit a joining report	Session
POST	<code>/api/ltc</code>	Submit an LTC claim	Session
POST	<code>/api/non-air-india</code>	Submit Air India exemption request	Session
POST	<code>/api/ex-india-leave</code>	Submit ex-India visit request	Session

Leave Management (`/api/leaves`)

Method	Endpoint	Purpose	Auth
GET	<code>/approvals</code>	Get pending approvals for current user	Session
POST	<code>/approvals/[applicationId]</code>	Approve or reject a specific application	Session
POST	<code>/approvals/bulk</code>	Bulk approve/reject multiple applications	Session
GET	<code>/my-submissions</code>	Get current user's submitted applications	Session
POST	<code>/acting-hod</code>	Manage acting HoD delegation	Session

Admin (`/api/admin`)

Method	Endpoint	Purpose	Auth
GET/POST	/users	List / create / bulk upload users (CSV)	Session (ADMIN)
GET	/statistics	Dashboard statistics (leave counts, dept breakdown)	Session (ADMIN)
GET	/audit	Query audit log with filters	Session (ADMIN)
GET	/application-trace	Trace a specific application through workflow	Session (ADMIN)
GET/POST	/form-templates	CRUD for JSON-schema form templates	Session (ADMIN)
GET/POST	/tasks	Manage workflow task instances	Session (ADMIN)

Forms ([/api/forms](#))

Method	Endpoint	Purpose	Auth
GET	/autofill	Pre-fill form fields from user profile	Session
POST	/submissions	Submit a dynamic form (template-based)	Session

Profile ([/api/profile](#))

Method	Endpoint	Purpose	Auth
GET	/	Get current user's profile data	Session

Station Leave Approvals ([/api/station-leave](#))

Method	Endpoint	Purpose	Auth
GET	/	Get station leave submissions	Session
POST	/approvals	Approve/reject station leave	Session
GET	/bootstrap	Initial data for station leave UI	Session

Design Decisions

- **All APIs are synchronous** — no async job queues; every request completes in a single HTTP roundtrip
- **Cookie-based session auth** — HMAC-SHA256 signed session tokens stored in [httpOnly](#), [secure](#), [sameSite: lax](#) cookies
- **Consistent error handling pattern** — every route catches [AuthError](#) (returns appropriate status) and generic [Error](#) (returns 400), with a fallback 400 for unknown errors
- **Audit logging on mutations** — every state-changing API logs to [AuditLog](#) with IP, user agent, and structured details

- **Fail-silent audit** — if audit logging fails, the main request still succeeds (audit must never block the user)

5. Approval Workflow Architecture

Flow: Leave Application → Multi-Step Approval → Final Status



Approval Chain by Role and Leave Type

Applicant Role	Leave Type	Approval Chain
Faculty	Earned Leave	HOD → Dean → Establishment
Faculty	Ex-India Visit	HOD → Dean → Director → Establishment
Faculty	LTC	HOD → Accounts → Establishment
Faculty	Station Leave	HOD (≤30 days) / HOD → Director (>30 days)
Staff	Earned/Casual	Reporting Officer → Registrar

Applicant Role	Leave Type	Approval Chain
HoD	Station Leave	Dean

Task Routing Engine

The workflow engine (`task-routing.ts`) supports five assignment modes:

Mode	Description
<code>specific</code>	Assign to specific user IDs
<code>role</code>	Assign to all active users with a given role
<code>department</code>	Assign to all active users in specified departments
<code>sameDepartmentRole</code>	Assign to users in the same department as the applicant with a specified role
<code>all</code>	Assign to all active users (broadcast)

Routing rules can be conditional per source role, enabling different chains for Faculty vs. Staff submissions.

Acting HoD Delegation

- When an HoD is on leave, an `ActingHodAssignment` record delegates approval authority to another faculty member
- Time-bounded (`startDate` → `endDate`) with full audit trail
- The acting HoD sees all pending approvals during the delegation period
- Assignment is tracked by `hodId`, `actingHodId`, and `assignedById`

Bulk Approvals

- Approvers can select multiple pending applications and approve/reject in one action
- `POST /api/leaves/approvals/bulk` processes each application sequentially
- OTP/signature verification happens once for the entire batch
- Individual audit events are logged per application

6. Authentication & Session Management

Architecture

```
User enters institute email
| POST /api/auth/request-otp
v
Server generates 6-digit OTP → bcrypt hash stored in OtpToken table
| Nodemailer sends styled HTML email via Gmail SMTP
v
User enters OTP
| POST /api/auth/verify-otp
v
Server verifies OTP against bcrypt hash
```

```
| Checks expiry (default 10 min) and attempt count
v
Creates HMAC-SHA256 session token: base64url(payload).signature
| Payload: { sub: userId, role: RoleKey, exp: timestamp }
| Signature: HMAC-SHA256(payload, SESSION_SECRET)
v
Sets httpOnly cookie: lf_session
| Config: httpOnly, secure (prod), sameSite: lax, maxAge: 12h
v
Redirects to role-specific dashboard
```

Session Token Format

```
<base64url(JSON payload)>.<HMAC-SHA256 signature>
```

```
Payload: {
  "sub": "cuid_user_id",
  "role": "FACULTY",
  "exp": 1724000000
}
```

- **No JWT library** — custom HMAC implementation using `node:crypto` for minimal dependencies
- **12-hour TTL** — configurable via `AUTH_SESSION_SECONDS` env var
- **Stateless verification** — no database lookup needed to validate the token; user lookup happens only in `requireSessionActor()`

OTP Security

Property	Value
OTP length	6 digits
Hash algorithm	bcrypt (salt rounds: 10)
Expiry	10 minutes (configurable via <code>OTP_EXP_MINUTES</code>)
Attempt tracking	<code>attempts</code> counter on <code>OtpToken</code> record
Fallback	If SMTP is not configured, OTP is logged to console (dev mode)

Cookie Security

Property	Value
<code>httpOnly</code>	<code>true</code> — not accessible via JavaScript
<code>secure</code>	Auto-detected: <code>true</code> in production with HTTPS, <code>false</code> in dev
<code>sameSite</code>	<code>lax</code> — prevents CSRF on cross-origin POST
<code>path</code>	<code>/</code>

Property	Value
<code>domain</code>	Configurable via <code>AUTH_COOKIE_DOMAIN</code>
<code>maxAge</code>	43200 seconds (12 hours)

Google OAuth (Optional)

- Client ID and secret configured via env vars
- `/api/auth/google` endpoint handles the OAuth flow
- Falls back to OTP if Google OAuth is not configured

Role-Based Access Control

Mechanism	Description
<code>requireSessionActor(token)</code>	Validates session, loads user with role and department from DB
<code>requireSessionActor(token, { roles: [RoleKey.ADMIN] })</code>	Validates session AND checks role — returns 403 if unauthorized
<code>requireSignedInForPage()</code>	Server Component helper — redirects to <code>/login</code> if not authenticated
<code>requireRoleForPage(role)</code>	Server Component helper — redirects to role-appropriate dashboard if wrong role

Multi-Tab Session Sync

- `AuthSessionSync` client component uses `BroadcastChannel` API to synchronize login/logout across browser tabs
- When one tab logs out, all tabs redirect to the login page

7. Digital Signature & OTP Verification

Overview

Every leave submission and approval action requires a digital signature verified by a fresh OTP. This creates a non-repudiable audit trail for each decision.

Signature Flow

1. User draws/types signature using `signature_pad` canvas component
2. User requests an OTP (sent to their institute email)
3. User enters the 6-digit OTP
4. Frontend sends signature image + OTP to the API
5. Server verifies OTP against bcrypt hash in `OtpToken` table
6. If valid, the action (submit/approve/reject) proceeds
7. Signature and verification metadata are stored in the application record

Signature Components

Component	Purpose
<code>signature-otp-verification-card.tsx</code>	Full signature + OTP verification card (27KB component)
<code>approval-signature-otp-modal.tsx</code>	Modal wrapper for approval signature flow
<code>bulk-approval-signature-otp-modal.tsx</code>	Batch approval signature verification
<code>hod-signature-approval-modal.tsx</code>	HoD-specific signature modal
<code>use-signature-otp.ts</code>	React hook encapsulating the OTP request/verify lifecycle

8. Audit System

Architecture

The audit system has two layers:

1. **Direct Audit Logging** (`logger.ts`) — records system events (login, submission, approval) with full request context
2. **Derived Audit Logs** (`derived-audit.ts`) — aggregates leave application submissions and approval decisions into a unified audit view

Direct Audit Logger

- Uses **random UUIDs** to avoid collisions during rapid bulk operations
- Captures **source IP** (with multi-header resolution: `x-forwarded-for`, `cf-connecting-ip`, `x-real-ip`, etc.)
- Captures **destination IP** (via DNS resolution with in-memory cache)
- Records **host IP** from `x-forwarded-server` header
- Every audit entry includes a `_source: "system-audit"` marker

IP Resolution

The audit system implements comprehensive IP extraction:

Header	Purpose
<code>request.ip</code>	Direct request IP (Next.js)
<code>Forwarded</code>	RFC 7239 <code>for=</code> parameter
<code>X-Forwarded-For</code> / <code>X-Vercel-Forwarded-For</code>	Proxy chain (first entry)
<code>X-Real-IP</code> / <code>CF-Connecting-IP</code> / <code>True-Client-IP</code> / <code>X-Client-IP</code>	Direct headers

IPv6-mapped IPv4 addresses (`::ffff:1.2.3.4`) are automatically normalized.

Database-Level Immutability

PostgreSQL triggers enforce audit log integrity:

- **AuditLog_block_manual_insert**: Only allows inserts where `details._source = 'system-audit'`
- **AuditLog_block_mutation**: Blocks all UPDATE and DELETE operations

Self-Healing Table

If the **AuditLog** table doesn't exist (e.g., fresh deployment), the logger auto-creates it with all indexes using `$executeRawUnsafe`. Audit logging **never blocks** the main request — failures are caught and logged to console.

9. Form System Architecture

Overview

The form system has two tiers:

1. **Hardcoded leave forms** — purpose-built pages for each leave type (Earned Leave, Station Leave, Ex-India, LTC, Non-Air India, Joining Report)
2. **Dynamic form templates** — admin-created JSON-schema forms rendered by a generic form renderer

Hardcoded Leave Forms

Each leave type has a dedicated page under `src/app/` with a specialized React component (~30-120KB each):

Form	Route	Size	Key Features
Earned Leave	<code>/earned-leave</code>	84KB	Date range, sessions, LTC toggle, reliever nomination
Station Leave	<code>/station-leave</code>	65KB	Routing panel, approval status history
LTC	<code>/ltc</code>	42KB	Family members, travel mode, fare estimates, Accounts verification
Ex-India Leave	<code>/ex-india-leave</code>	—	Itinerary, host institute, Form I & II attachments
Non-Air India	<code>/non-air-india</code>	—	Fare comparison, MHRD permission, airline selection
Joining Report	<code>/joining-report</code>	—	Rejoining date, office order reference, leave category

Dynamic Form Builder

The admin panel includes a full form builder (`admin-form-builder.tsx`, 137KB) that:

- Defines form schemas as JSON
- Supports field types: text, number, date, select, checkbox, textarea, file upload
- Configures multi-step workflow with task routing rules
- Publishes templates that become available to users
- Tracks submissions with step-by-step workflow progression

Form Rendering

`template-form-renderer.tsx` (36KB) is a generic renderer that:

- Parses the JSON schema from `FormTemplate.schema`
- Renders fields dynamically using `react-hook-form`
- Validates using Zod schemas generated from the template
- Handles workflow step progression and task assignments

Auto-Fill System

- `form-autofill.ts` pre-populates form fields from the user's profile (name, email, department, designation)
- `leave-session.ts` manages session-level form state persistence

Client-Side Data Protection

- `encrypted-local-storage.ts` encrypts auto-saved form data using **AES-256-GCM** with **PBKDF2** key derivation (120,000 iterations)
- Encryption key is generated per browser session and stored in `sessionStorage`
- Ensures sensitive leave data (medical reasons, personal details) is not stored in plaintext in `localStorage`

10. PDF Export

Architecture

Client-side PDF generation using html2canvas + jsPDF:

```
HTML form element
|
v html2canvas (scale: 1.5x)
Render to canvas (with sanitization)
|
|-- Remove SVGs (oklab/oklch color parsing issues)
|-- Replace form fields with text spans
|-- Normalize colors to safe ink/border values
|-- Strip modern CSS (oklab, oklch, lab, lch functions)
|
v jsPDF
Slice canvas into A4 pages
|
|-- 24pt margins
|-- JPEG encoding (82% quality)
|-- Multi-page support (auto-pagination)
|
v
Download as PDF
```

Sanitization

The PDF export sanitizes the DOM clone to ensure `html2canvas` compatibility:

- Removes all `<svg>` elements (avoids color parsing failures)
- Replaces `<input>`, `<textarea>`, `<select>` with `<span>` showing the value
- Strips modern color functions (oklab, oklch, lab, lch)
- Forces white background with safe ink colors
- Includes a **fallback scale** (1x) if the primary render (1.5x) fails

11. Scalability

Current Architecture Bottlenecks

Users	Frontend	API Routes	Database	Email
10	Fine	Fine	Fine	Fine
100	Fine	Fine	Fine	SMTP rate limits possible
500	Fine	Single Node.js process saturated	Connection pool (default 5) may exhaust	Gmail SMTP throttling likely
1,000+	Fine (SSR cached)	Bottleneck — needs multiple instances	Needs connection pooling tuned	Needs dedicated SMTP service

Scaling Strategy

Technique	Current Status	What's Needed
Horizontal scaling	Single Next.js instance	Vercel auto-scales serverless functions; or deploy multiple containers behind LB
Database pooling	PgBouncer via Supabase (port 6543)	Already configured; increase pool size for higher concurrency
Caching	No caching layer	Add Redis for session validation, role lookups, form template caching
Email queue	Synchronous Nodemailer calls	Queue email sending via background jobs (BullMQ / Vercel Edge Functions)
Static generation	All pages are SSR	Pre-render public pages; use ISR for semi-static content
CDN	Vercel Edge Network	Already leveraged if deployed on Vercel

12. Performance

Expected Latency

Operation	Expected Latency	Bottleneck
Login page load (SSR)	200-400ms	Server-side render + Supabase connection
OTP request	500-2000ms	SMTP send time (Gmail)
OTP verification	100-300ms	bcrypt compare + Prisma query
Dashboard load (SSR)	300-800ms	Multiple Prisma queries with joins
Leave form submission	200-500ms	Prisma transaction + audit log + email send
Approval action	300-600ms	OTP verify + Prisma update + email notification
PDF export	1-5s	Client-side html2canvas rendering
Admin audit log query	500-2000ms	Derived audit aggregation across tables

Likely Bottlenecks

- 1. **SMTP email sending** — synchronous in the request path; Gmail may throttle under load
- 2. **Derived audit queries** — merges two table scans with text search
- 3. **Large form components** — earned leave page is 84KB, station leave approvals 65KB; bundle size impacts initial load
- 4. **Admin form builder** — 137KB component; heavy client-side rendering

Optimization Strategies

- **Move email sending to background** — fire-and-forget after the main response
- **Add pagination** to all list endpoints (audit logs, submissions, users)
- **Code-split** large form components with `next/dynamic`
- **Cache role and department lookups** — these rarely change
- **Add database indexes** for frequently queried combinations (e.g., `[applicantId, status, createdAt]`)

13. Reliability

Failure Scenarios and Current Handling

Failure	Current Behavior	Ideal Behavior
Database down	Prisma throws connection error → API returns 400/500	Retry with exponential backoff; health check endpoint
SMTP down	<code>sendOtpEmail</code> throws → user sees error	Graceful fallback: show "email delivery delayed" message; retry queue
Session expired	<code>AuthError</code> with status 401 → redirect to login	Already handled; auto-redirect on all protected pages
Audit log write fails	Silently caught → main request succeeds	Already designed: audit never blocks the user

Failure	Current Behavior	Ideal Behavior
OTP table missing	Auto-created by Prisma migration	Already handled via schema migration
AuditLog table missing	Auto-created by <code>ensureAuditLogTable()</code> with raw SQL	Already self-healing

What's Missing

Concept	Status
Health check endpoint	None — no <code>/api/health</code> route
Request timeouts	No explicit timeout on Prisma queries
Circuit breakers	None — every request hits the database
Retry logic	No retry on email or database failures
Idempotency	No idempotency keys — duplicate form submissions create duplicate applications
Rate limiting	No API rate limiting — relies on session auth only

14. Security

Authentication & Authorization

- **OTP-only auth:** No passwords stored; bcrypt-hashed 6-digit OTPs with configurable expiry
- **HMAC session tokens:** Custom implementation using `node:crypto` HMAC-SHA256; no JWT library dependency
- **Cookie security:** `httpOnly`, `secure` (production), `sameSite: lax`, configurable domain
- **Role-based access:** Server-side checks on every API route and page via `requireSessionActor()`
- **Reporting chain enforcement:** Approval steps are only assignable to users in the correct role

Client-Side Security

- **Encrypted localStorage:** Form auto-save data encrypted with AES-256-GCM (PBKDF2 key derivation, 120K iterations)
- **Session-scoped encryption key:** Key stored in `sessionStorage`, lost on tab close
- **Multi-tab sync:** Logout propagated across tabs via `BroadcastChannel`

API Security

- **Session cookie auth on all mutating endpoints:** No public write endpoints
- **Error masking:** Generic error messages returned to client; stack traces only in development
- **Input validation:** Zod schemas validate all form inputs before database operations

Audit & Compliance

Measure	Implementation
Immutable audit log	PostgreSQL triggers block UPDATE/DELETE on <code>AuditLog</code>
Insert guard	Only <code>_source: "system-audit"</code> entries accepted
IP forensics	Full request IP chain captured (source, destination, host)
Digital signatures	OTP-verified signatures on every submission and approval
Non-repudiation	Each action tied to a verified user identity + IP + timestamp

Known Security Gaps

- **No rate limiting:** No protection against brute-force OTP guessing (mitigated by short expiry)
- **No CSP headers:** Missing Content-Security-Policy
- **Session secret in env:** `AUTH_SECRET` has a hardcoded fallback (`dev-session-secret-change-me`) — must be overridden in production
- **No CSRF tokens:** Relies on `sameSite: lax` cookie + session auth
- **OTP randomness:** Uses `Math.random()` for OTP generation — should use `crypto.randomInt()` for cryptographic security
- **Sensitive data in .env:** Database credentials and SMTP passwords in `.env` file (`.gitignore`'d but no vault integration)

15. Deployment / Infrastructure

Current Deployment

Component	Platform	Configuration
Application	Vercel (or local <code>next dev</code>)	Next.js 16 with App Router
Database	Supabase (PostgreSQL)	PgBouncer pooling on port 6543; direct connection on port 5432
Email	Gmail SMTP	Port 587, TLS, app-specific password
DNS/CDN	Vercel Edge Network	Automatic if deployed on Vercel

Environment Variables

Variable	Purpose
<code>DATABASE_URL</code>	PostgreSQL connection string (pooled via PgBouncer)
<code>DIRECT_URL</code>	Direct PostgreSQL connection (for migrations)
<code>NEXT_PUBLIC_APP_URL</code>	Public URL used in emails and redirects
<code>EMAIL_SERVER_HOST</code> / <code>PORT</code> / <code>USER</code> / <code>PASSWORD</code>	SMTP configuration
<code>EMAIL_FROM</code>	Sender email address

Variable	Purpose
OTP_EXP_MINUTES	OTP expiry duration
GOOGLE_CLIENT_ID / SECRET	Google OAuth credentials
AUTH_SECRET	HMAC signing key for session tokens
AUTH_SESSION_SECONDS	Session TTL (default 43200 = 12h)
AUTH_COOKIE_DOMAIN	Cookie domain override
AUTH_COOKIE_SECURE	Force secure cookie flag
AUTH_ENFORCE_CANONICAL_HOST	Enable canonical URL redirect middleware

Middleware

- `proxy.ts` implements canonical host redirect middleware — if `AUTH_ENFORCE_CANONICAL_HOST=true`, non-canonical requests receive a 308 redirect
- Bypasses `/_next`, `/favicon`, `/robots.txt` paths

CI/CD

- **Husky** pre-commit hooks run `lint-staged` (ESLint + Prettier)
- **No CI pipeline** — no automated tests or build verification
- **Prisma migrations** managed via `prisma migrate dev` (local) and `prisma migrate deploy` (production)

Database Seeding

- `prisma/seed.ts` creates a full demo environment: 4 departments, 10 roles, 11 users (all roles), 6 leave types, 1 sample leave application with 3 approval steps, and 2 notifications
- Uses IIT Ropar email aliases (`2023csb1288+slug@iitrpr.ac.in`) for test accounts

16. Current Limitations

Category	Limitation
Auth	OTP uses <code>Math.random()</code> instead of <code>crypto.randomInt()</code> for generation
Rate limiting	No API rate limiting — vulnerable to abuse on OTP and submission endpoints
Email	Synchronous email sending in the request path; Gmail SMTP has daily limits
Real-time	No WebSockets — approval status requires page refresh to update
Testing	No automated tests (unit, integration, or E2E)
Monitoring	No APM, error tracking, or alerting; <code>console.log</code> only
Offline	No service worker or offline capability
Mobile	Responsive CSS but no native mobile app

Category	Limitation
File uploads	No file upload infrastructure (attachments referenced but upload endpoint not implemented)
Search	No full-text search; contains queries on PostgreSQL
Scalability	Single Next.js instance; no caching layer
Idempotency	Duplicate form submissions create duplicate leave applications

17. Design Alternatives

Framework

Option	Pros	Cons	Verdict
Next.js 16 (current)	SSR, API routes, single deployment, React Server Components	Vendor lock-in (Vercel), large bundle, opinionated routing	Best for this scale; server-rendered institutional portal
Express + React SPA	Full control, separate concerns	Two deployment units, duplicate types, CORS complexity	Overkill for a single-team project
Django	Built-in admin, ORM, forms	Python ecosystem, different frontend stack	Good alternative for form-heavy apps

Database

Option	Pros	Cons	Verdict
PostgreSQL + Prisma (current)	Relational integrity, typed queries, migrations	Prisma abstraction overhead, raw SQL for triggers	Best fit: approval chains need joins and constraints
MongoDB	Flexible schema, embedded documents	No relational integrity for approval chains	Poor fit: hierarchical workflows need FK constraints
MySQL	Widespread, mature	Less JSON/JSONB support	Viable but PostgreSQL's JSONB is more flexible for metadata

Auth

Option	Pros	Cons	Verdict
Custom OTP + HMAC (current)	Full control, minimal deps, passwordless	Custom security code, no library community	Good fit: OTP-only is simple enough to implement correctly

Option	Pros	Cons	Verdict
NextAuth.js	Provider abstraction, session management	Heavy for OTP-only; complex configuration	Overkill for single-provider OTP auth
Clerk / Auth0	Managed service, MFA built-in	External dependency, cost, vendor lock-in	Better if budget allows and compliance requires MFA

Email

Option	Pros	Cons	Verdict
Nodemailer + Gmail (current)	Free, simple setup	Daily send limits, synchronous, less reliable	Fine for prototype; needs upgrade for production
SendGrid / Resend	Managed delivery, templates, analytics	Cost, external dependency	Better for production scale
AWS SES	Low cost, high throughput	AWS infrastructure setup	Best for high-volume email

18. Important SDE Concepts Applied

Concept	Where Applied
Role-Based Access Control (RBAC)	10 distinct roles with hierarchical permissions; server-side enforcement on every route
Workflow Engine	Sequential approval chains with configurable routing rules per role and leave type
Immutable Audit Log	PostgreSQL triggers prevent modification; UUIDs prevent collision; full IP forensics
Self-Referential Relations	User → reportsTo → User creates the organisational hierarchy
Delegation Pattern	ActingHodAssignment enables temporary authority transfer with time bounds
Schema-Driven Forms	JSON schema templates enable admin-created forms without code changes
Client-Side Encryption	AES-256-GCM with PBKDF2 key derivation for localStorage data protection
Canonical URL Middleware	308 redirects enforce single canonical hostname
Self-Healing Infrastructure	AuditLog table auto-created if missing; triggers re-installed on each startup
Type Safety End-to-End	Prisma types → server logic → API routes → React components all share TypeScript types

Concept	Where Applied
Database Migrations	Prisma Migrate for versioned, reproducible schema changes
Pre-Commit Quality Gates	Husky + lint-staged enforce ESLint + Prettier on every commit
Separation of Concerns	Clear layering: <code>app/</code> (pages) → <code>server/</code> (business logic) → <code>modules/</code> (config) → <code>lib/</code> (utilities)
Cookie Security Best Practices	httpOnly, secure, sameSite, configurable domain, proper max-age

19. Failure Scenarios

Scenario 1: 500 Users Submit Leave Applications During Semester End

- Multiple faculty submit earned leave simultaneously at semester break
- All requests hit `POST /api/earned-leave` → Prisma creates `LeaveApplication` + `ApprovalSteps`
- Each submission triggers a synchronous `sendLeaveSubmissionEmail()` call
- **Gmail SMTP throttles** at ~500 emails/day → email send failures
- API returns errors even though the leave was created successfully
- **Fix:** Decouple email from the request path; use a queue (BullMQ) or fire-and-forget

Scenario 2: HoD is on Leave With No Acting HoD Assigned

- Faculty submits earned leave → `ApprovalStep` assigned to HoD
- HoD is absent with no `ActingHodAssignment` active
- Application sits at PENDING indefinitely — no escalation mechanism
- **Fix:** Add configurable SLA timers with auto-escalation to Dean after N days

Scenario 3: Database Connection Pool Exhaustion

- Multiple concurrent requests → Prisma default connection pool (5) saturated
- New requests wait for connections → timeouts cascade
- Supabase PgBouncer helps but has its own pool limits
- **Fix:** Tune `connection_limit` in `DATABASE_URL`; add connection monitoring; implement request queuing

Scenario 4: Audit Log Table Corrupt or Dropped

- If `AuditLog` table is dropped or schema changes, Prisma throws `P2021` (table not found)
- Logger catches the error → calls `ensureAuditLogTable()` → auto-creates table with indexes and triggers
- Retries the audit insert
- **Impact:** One failed audit log on first occurrence; self-heals for subsequent requests

Scenario 5: SMTP Credentials Expire

- Gmail app password revoked → Nodemailer throws on `sendOtpEmail()`
- Users cannot log in (OTP not delivered)

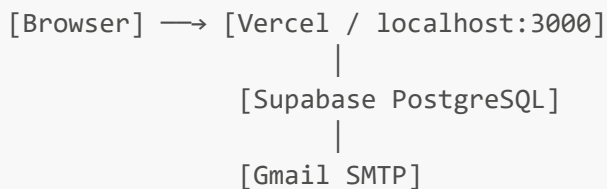
- **Graceful degradation:** If `emailTransportConfigured` is false, OTP is logged to console (dev mode only)
- **Production impact:** Complete auth failure — no login possible
- **Fix:** Health check endpoint that tests SMTP connectivity; alert on failure

Scenario 6: Duplicate Form Submission (Double Click)

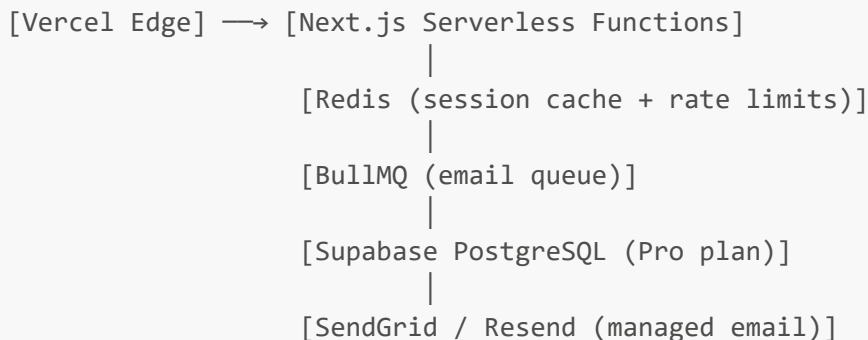
- User clicks "Submit" twice rapidly
- Two `POST /api/earned-leave` requests arrive
- No idempotency check → two `LeaveApplication` records created with different reference codes
- **Fix:** Add client-side submit debouncing (already partially present); add server-side idempotency key

20. Future Architecture

Phase 1: Current (1-50 users)

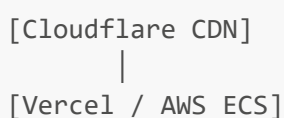


Phase 2: Medium Scale (50-500 users)



- Add Redis for session caching and distributed rate limiting
- Queue email sending with BullMQ
- Upgrade Supabase to Pro plan for higher connection limits
- Add rate limiting on OTP and submission endpoints
- Add health check endpoint

Phase 3: Production Scale (500+ users)



```
      |  
[Next.js instances xN] ← [Redis Cluster (ElastiCache)]  
      |  
[PostgreSQL (RDS / Supabase Pro)]  
      |  
[SQS / BullMQ (email + notification queue)]  
      |  
[SES (email)] + [SNS (push notifications)]  
      |  
[Prometheus + Grafana (monitoring)]  
[Sentry (error tracking)]
```

- Container-per-request with auto-scaling
- PostgreSQL read replicas for reporting queries
- Full observability stack (Prometheus, Grafana, Sentry)
- CI/CD pipeline with automated tests, staging environment
- Mobile-responsive PWA or native companion app
- Full-text search with Elasticsearch for audit log queries
- Webhook notifications for external system integration